

Novosibirsk State University

Backgammon
Technical Specification
Digital Platforms course project

Group 25214
Dedov Dmitry
Starikova Maria
Medvedev Mikhail

1. Introduction.....	3
2. Hardware Requirements.....	3
2.1 Game board.....	3
2.2 Matrix controllers.....	3
2.3 Memory Wrapper.....	3
2.4 Turn Controller.....	4
2.5 Dice Displays.....	4
2.6 I/O.....	4
2.6.1 TTY.....	4
2.6.2 Keyboard.....	5
2.6.3 I/O Contollers.....	5
3. Software Requirements.....	5
3.1 I/O Functions.....	5
3.2 Game Initialization.....	5
3.3 Move Validation.....	6
3.4 Player move.....	7
3.5 Computer Move.....	7
3.6 Computer Move Evaluation.....	8
3.6 Bear off.....	8
3.7 Random.....	9

1. Introduction

The main objective of the project is to implement a backgammon board game using a sixteen-bit CdM-16 processor and a circuit implemented in the Logisim program.

2. Hardware Requirements

2.1 Game board

The game field visualization must be implemented on a system of multiple matrices. The playing field layer consists of two matrices with 25x20 pixels resolution, which form a matrix 25x40 pixels resolution. This layer is responsible for displaying the static board background. Two additional matrices, each 25x32, must be placed on top of the first layer to render the checkers. The 25x32 size allows for displaying high stacks of checkers in each point. The system must render a standard 24-point field for long backgammon.

2.2 Matrix controllers

To interact with the game field, the matrices that render checkers must have special controllers. Each controller contains 24 registers, each storing a value from 0 to 11 (representing the checker count at that field for which this register is responsible).

There must be two such controllers: one for the white checkers layer and one for the black checkers layer.

2.3 Memory Wrapper

To speed up the project, a memory wrapper must be implemented. This element allows us to connect memory and matrix controllers. The wrapper

must monitor memory addresses where the number of checkers for each field is stored. When the CPU changes a value in the matrix state memory, this element must automatically detect changes and send a signal to matrix controllers. This removes the need for the CPU to manually control every pixel on the display.

2.4 Turn Controller

The main task of the module is to monitor the progress of the game and signal only to a certain matrix, Depending on which player is currently moving, the controller must send a "Write Enable" signal only to the corresponding matrix controller (white or black). Also, the controller must be in a neutral state where no signal is sent to any of the matrices

2.5 Dice Displays

Two separate displays must be used to show the dice values. Each display must have its own controller. These controllers must independently monitor the memory cells where the CPU stores the dice roll results. When a player changes or a new roll occurs, the controllers must update the information on the displays. The dice displays must work separately from the main game board matrices.

2.6 I/O

2.6.1 TTY

A TTY terminal must be used to display the text log of the game. It must show information about the current turn, and messages about player movements. If a player tries to make an illegal move, the terminal must display an error message. The terminal is the main element that provides the player with information about the current state of game.

2.6.2 Keyboard

The primary input device for the player is the keyboard. It must allow the player to enter coordinates to select a checker and the destination point for the move. The input from the keyboard must be processed by the CPU, checked by the game rules, and then change matrix state memory.

2.6.3 I/O Controllers

For the CPU and terminal to interact with the keyboard, each element must have a controller embedded in memory at a specific address so that data can be read and recorded at these addresses in the program. This means the keyboard and the terminal (TTY) are linked to dedicated cells in the RAM. The program interacts with these devices by reading from or writing to these specific addresses.

3. Software Requirements

3.1 I/O Functions

Two functions are used to output the game state and input the player's moves: print to tty and get char. These functions interact directly with the memory addresses assigned to the TTY terminal and the keyboard.

3.2 Game Initialization

On start program must correctly initialize the playing field before starting the game. At the beginning, the system must place 11 checkers at the starting positions (the "Head") for both the player and the opponent. All other points on the board must be cleared. After the program starts the main game loop, which continuously checks for user input and manages the turn sequence.

3.3 Move Validation

To ensure the game follows the rules, the program has a specialized validation function. This function checks every move entered by the player before it is executed. If a move violates any rule, the system blocks it and displays an error message on the TTY terminal.

The validation process consists of the following mandatory checks:

1. **Ownership Check:** The program verifies that the selected checker belongs to the current player. A player cannot move the opponent's checkers.
2. **Point Availability:** The system checks the destination point. In Long Backgammon, a move is impossible if the target point is already occupied by the opponent's checkers.
3. **Distance and Dice Match:** The program calculates the distance between the starting point and the target point. This distance must match one of the values available on the dice. If the distance does not match any dice result, the move is rejected.
4. **Head Rule:** The program monitors moves from the starting position. According to the rules, only one checker can be taken from the "Head" per turn. The system uses a special flag to track this and blocks the move if the player tries to take a second checker.
5. **Prime rule (zabor):** The program must implement a special check for creating a "fence" or a block of six checkers. The software must monitor the creation of six consecutive occupied points. The rules of Long Backgammon state that a player cannot build a sequence of six checkers in a row if the opponent has no chips ahead of this block. This prevents one player from completely blocking the opponent's movement.

3.4 Player move

The player move function manages the entire sequence of a single turn. At the start of the turn, the system rolls two dice using the randomize functions. The program checks for "doubles" (when both dice have the same value).

Normal Roll: The player receives 2 moves, and only 1 checker can be taken from the "Head".

Double Roll: The player receives 4 moves of the same value, and the "Head Rule" is allowed to take 2 checkers.

The dice values are stored in a memory mapped array for display and in a local array in the program to track used moves.

The program enters a loop that runs while dice_count is greater than zero. In each iteration. The TTY terminal shows the number of moves left. The system fetches coordinates from the keyboard. The player enters the "From" and "To" positions. Validation is required for every move. If valid, the checkers position is updated in memory and in matrix. Before each move, the program checks the position of all checkers. The system will switch to 'bear off' mode if all checkers are in the 'home' zone. In this mode, the player can remove checkers from the board instead of moving them to another point. This is necessary to finish the game. The system also has a "Pass" function. It is used if the player has no legal moves or wants to stop the turn before using all dice. When this function is called, the program stops the input loop and transfers the turn to the next player.

3.5 Computer Move

The computer move function implements the logic for the automated opponent. The board's current state is analyzed by the computer and the available cube values are used to select moves. The computer uses a greedy strategy to make decisions. For each available die, the program scans all 24

points on the board. If a point contains the computer's checkers, the system calculates a potential move and checks its legality. Once the optimal move is identified, the computer executes it. The program updates the checker positions in the RAM, updates the dice display for the user, and reduces the `dice_count`. If the AI completes its moves or finds no legal options, it prints "Computer passed" to the terminal and transfers control back to the player.

3.6 Computer Move Evaluation

The move evaluation function calculates the numerical value of a move. The system uses this number to determine the highest priority move for the computer. The function returns an integer – the estimated value of the move based on the current board state. The function analyzes the start and end points of the move and assigns a score based on several criteria. These criteria include the position of the checkers on the board, the safety of the end point, and the importance of certain areas of the board. For each checker, a certain value is added or subtracted from the total score, forming the final score for the current move being processed. The result of this function is used by the computer's core logic to select the final move. After searching for all valid moves, the system compares their scores. The move with the highest total score is selected and executed. If multiple moves have the same score, the program selects the first one found in the search cycle.

3.6 Bear off

The bear off validation function is responsible for the final stage of the game. It determines if a player can legally remove a checker from the board. This function is triggered only when all checkers are in the "home" zone and ensures that the dice values are used correctly according to the rules of Long Backgammon.

First, the function verifies that the selected point contains the player's checkers. Then, it calculates the distance required for the checker to exit the

board. For the first player, the distance is calculated relative to point 24, and for the second player, it is relative to point 12. This distance represents the exact dice value needed to remove the checker.

The program scans the available dice to find an exact match for the exit distance. If an exact match is found, the move is confirmed. If not, the program performs a "checkers behind" scan. It checks all points in the home zone that are further from the exit than the current point. If there is at least one checker behind, the player cannot use a larger die to remove a checker from the current point.

If there are no checkers behind the current point, the "Senior Dice Rule" is applied. This rule allows a player to use a die value that is greater than the distance to the exit. The algorithm searches for the smallest available die that is still larger than the required distance. If such a die is found, it is used to remove the checker, and the move is considered legal.

3.7 Random

The system must implement a pseudo-random number generator to produce dice values in the range of 1 to 6.